3.5 Pooling

- often, as we process images, we want to gradually reduce the spatial resolution of our hidden representations, aggregating information so that the higher up we go in the network, the larger the receptive field (in the input) to which each hidden node is sensitive
- often, our ultimate task asks some global question about the image, e.g., *does it contain a cat*?
- so, typically, the units of our final layer should be sensitive to the entire input
- by gradually aggregating information, yielding coarser and coarser maps, we accomplish this goal of ultimately learning a global representation, while keeping all of the advantages of convolutional layers at the intermediate layers of processing

3.5 Pooling

- moreover, when detecting lower-level features, such as edges, we often want our representations to be somewhat invariant to translation
- for instance, if we take the image $\mathbf{X}$, with a sharp delineation between black and white, and shift the whole image by one pixel to the right, i.e., $[\mathbf{Z}]_{i,j} = [\mathbf{X}]_{i,j+1}$, then the output for the new image $\mathbf{Z}$ might be vastly different
- the edge will have shifted by one pixel; in reality, objects hardly ever occur exactly at the same place
- in fact, even with a tripod and a stationary object, vibration of the camera due to the movement of the shutter might shift everything by a pixel or so (high-end cameras are loaded with special features to address this problem)

3.5 Pooling

- this section introduces *pooling layers*, which serve the dual purposes of mitigating the sensitivity of convolutional layers to location and of spatially downsampling representations
- like convolutional layers, *pooling* operators consist of a fixed-shape window that is slid over all regions in the input according to its stride, computing a single output for each location traversed by the fixed-shape window (sometimes known as the *pooling window*)
- however, unlike the cross-correlation computation of the inputs and kernels in the convolutional layer, the pooling layer contains no parameters (there is no *kernel*)

3.5 Pooling

- instead, pooling operators are deterministic, typically calculating either the maximum or the average value of the elements in the pooling window
- these operations are called *maximum pooling* (*max-pooling*, for short) and *average pooling*, respectively
- in both cases, as with the cross-correlation operator, we can think of the pooling window as starting from the upper-left of the input tensor, and sliding across the input tensor from left to right and top to bottom
- at each location that the pooling window hits, it computes the maximum or average value of the input subtensor in the window, depending on whether max or average pooling is employed

3.5 Pooling

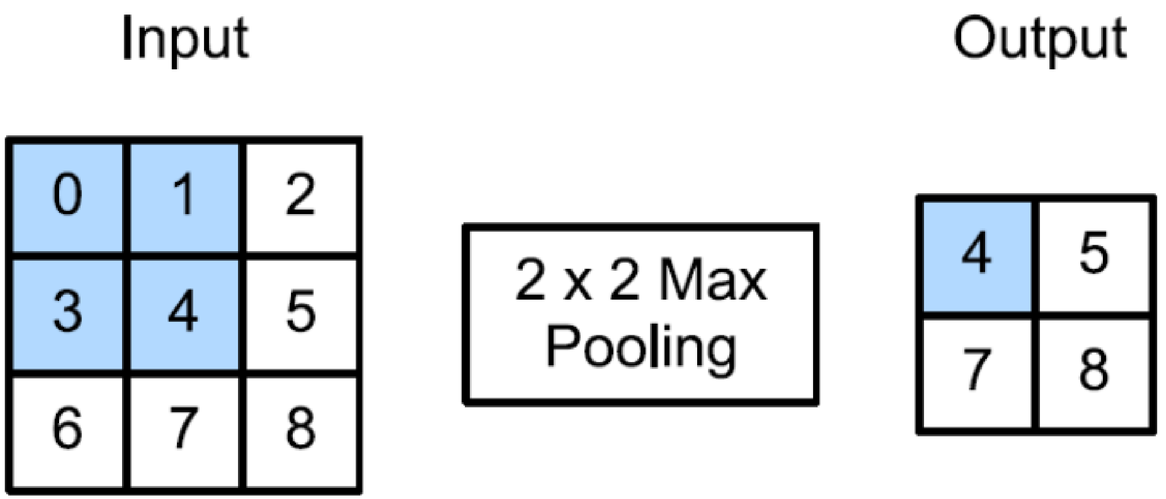

Figure 8: Maximum pooling with a pooling window shape of 2×2. The shaded portions are the first output element, as well as the input tensor elements used for the output computation: $\max(0, 1, 3, 4) = 4$.

3.5 Pooling

- the output tensor in Figure 8 has a height of 2 and a width of 2
- the four elements are derived from the maximum value in each pooling window:

$$\max(0, 1, 3, 4) = 4,$$
$$\max(1, 2, 4, 5) = 5,$$
$$\max(3, 4, 6, 7) = 7,$$
$$\max(4, 5, 7, 8) = 8.$$

- a pooling layer with a pooling window shape of $p \times q$ is called a $p \times q$ *pooling layer*
- the pooling operation is called $p \times q$ *pooling*

3.5 Pooling

- let us return to the object edge detection example mentioned at the beginning of this section
- now, we will use the output of the convolutional layer as the input for 2×2 maximum pooling
- set the convolutional layer input as $\mathbf{X}$ and the pooling layer output as $\mathbf{Y}$
- whether or not the values of $[\mathbf{X}]_{i,j}$ and $[\mathbf{X}]_{i,j+1}$ are different, or $[\mathbf{X}]_{i,j+1}$ and $[\mathbf{X}]_{i,j+2}$ are different, the pooling layer always outputs $[\mathbf{Y}]_{i,j} = 1$
- that is to say, using the 2×2 maximum pooling layer, we can still detect if the pattern recognized by the convolutional layer moves no more than one element in height or width

3.5 Pooling

- as with convolutional layers, pooling layers can also change the output shape
- and, as before, we can alter the operation to achieve a desired output shape by padding the input and adjusting the stride
- when processing multi-channel input data, the pooling layer pools each input channel separately, rather than summing the inputs up over channels, as in a convolutional layer
- this means that the number of output channels for the pooling layer is the same as the number of input channels